

Faxinator

Sistema de Agendamento de Serviços de Limpeza

Documentação do Projeto

Lista de Requisitos Funcionais

Descritivo de Casos de Teste de Software

Lista de Requisitos de Infraestrutura

Sumário

1. Contextualização e Escopo

2. Requisitos de Infraestrutura

2.1. Ambiente de desenvolvimento

2.2. Dependências do projeto

3. Arquitetura da Solução

3.1. Visão geral

3.2. Organização do backend

3.3. Organização do frontend

3.4. Decisões técnicas relevantes

4. Modelo de Dados

5. Lista de Requisitos Funcionais

6. Descritivo de Casos de Teste de Software

6.1. Ferramentas utilizadas

6.2. Ambiente de teste

6.3. Massa de dados

6.4. Casos de teste

6.5. Resumo dos resultados

6.6. Defeitos encontrados e corrigidos

7. Instruções de Execução

8. Credenciais de Acesso

1. Contextualização e Escopo

Uma empresa de serviços de limpeza oferece agendamentos de faxinas residenciais e comerciais, mas enfrenta dificuldades na gestão desses agendamentos por não possuir um sistema informatizado para controlar datas e horários das solicitações. A ausência de controle resulta em agendamentos duplicados, falha de comunicação entre funcionários e clientes, e dificuldade em organizar a disponibilidade dos profissionais.

O sistema incorpora um mecanismo de controle de agendamentos que emite alertas automáticos em duas situações distintas: quando há *conflito de horário* com outro serviço já atribuído ao mesmo profissional, e quando o profissional *não possui disponibilidade cadastrada* para o dia e horário solicitados. Além disso, mantém um histórico completo de cada operação, identificando o cliente, o profissional responsável, o operador que executou a ação e a data da operação, garantindo rastreabilidade e transparência dos processos.

Entregas cobertas por este documento

Entrega	Descrição	Localização
1	Lista de requisitos funcionais	Seção 5 deste documento
2	Diagrama Entidade-Relacionamento (DER)	Arquivo de imagem em der/ (entrega separada)
3	Script de criação e população do banco	backend/db/faxina_db.sql
4	Código-fonte do sistema	backend/ e frontend/
8	Descritivo de casos de teste de software	Seção 6 deste documento
9	Lista de requisitos de infraestrutura	Seção 2 deste documento

2. Requisitos de Infraestrutura

2.1. Ambiente de desenvolvimento

Item	Tecnologia	Versão
SGBD	PostgreSQL	18.3 (PostgreSQL 18.3 on x86_64windows, compiled by msvc-19.44.35225, 64-bit)
Linguagem de programação	TypeScript (backend)	5.9.3
	TypeScript (frontend)	6.0.3
	Node.js (ambiente de execução)	v24.11.1
	npm (gerenciador de pacotes)	11.12.1
Sistema operacional	Microsoft Windows 11 Pro	10.0.26200.8973

2.2. Dependências do projeto

Backend — API REST responsável pela regra de negócio e acesso ao banco de dados.

Pacote	Versão	Finalidade
express	5.2.1	Framework HTTP: roteamento, middlewares e tratamento de requisições
pg	8.23.0	Driver PostgreSQL, com consultas parametrizadas e pool de conexões
bcrypt	6.0.0	Geração e verificação de hash de senha (custo 12)
jsonwebtoken	9.0.3	Assinatura e verificação do token de acesso (JWT)
cookie-parser	1.4.7	Leitura do cookie httpOnly que transporta o token
cors	2.8.5	Liberação de origem cruzada com envio de credenciais
dotenv	17.3.1	Carregamento das variáveis de ambiente do arquivo .env
tsx	4.21.0	Execução direta de TypeScript em desenvolvimento

Frontend — Interface web (SPA) consumida pelo operador.

Pacote	Versão	Finalidade
react / react-dom	19.2.8	Biblioteca de construção da interface
vite	8.2.1	Servidor de desenvolvimento e empacotador
react-router	8.3.0	Roteamento entre as telas da aplicação
axios	1.19.0	Cliente HTTP para consumo da API, com envio de cookies

Pacote	Versão	Finalidade
tailwindcss	4.3.3	Estilização utilitária das interfaces
react-toastify	11.1.0	Notificações de sucesso das operações
react-icons	5.7.0	Conjunto de ícones

Requisitos mínimos para execução: PostgreSQL 18 ou superior instalado e em execução; Node.js 20 ou superior; navegador web moderno (Chrome, Edge ou Firefox em versão atual); portas 5000 (API) e 3000 (interface) livres.

3. Arquitetura da Solução

3.1. Visão geral

O sistema é dividido em dois projetos independentes que se comunicam por uma API REST:

- **backend** — API REST em Node.js com Express, acessando o PostgreSQL diretamente pelo driver `pg`, sem ORM. Concentra toda a regra de negócio, as validações e as verificações de conflito e disponibilidade.
- **frontend** — aplicação React de página única, responsável apenas pela apresentação e pela captura de dados do operador.

A autenticação usa **JWT transportado em cookie httpOnly** com expiração de 1 hora. Como o cookie httpOnly não é acessível por JavaScript, o frontend descobre o usuário autenticado consultando o endpoint `GET /me` ao carregar a aplicação.

3.2. Organização do backend

O backend segue o padrão **MVC em camadas**, com responsabilidades separadas e injeção de dependência por construtor:

Camada	Diretório	Responsabilidade
Model	<code>src/models/</code>	Interfaces TypeScript que descrevem o formato das linhas das tabelas e os objetos de transferência de dados
Repository	<code>src/repositories/</code>	Único ponto de acesso ao banco. Consultas SQL parametrizadas e controle de transação
Service	<code>src/services/</code>	Regra de negócio: validações, verificação de conflito e de disponibilidade, ordenação
Controller	<code>src/controllers/</code>	Tradução entre HTTP e domínio: leitura de parâmetros, montagem da resposta
Router	<code>src/routes/</code>	Declaração dos endpoints e associação com os controllers
Middleware	<code>src/middleware/</code>	Autenticação por cookie e tratamento centralizado de erros
Utils	<code>src/utils/</code>	Funções de apoio: hash, JWT, data/hora e algoritmo de ordenação

3.3. Organização do frontend

Diretório	Conteúdo
<code>src/pages/</code>	Uma pasta por tela: Login, Home, AppointmentRegistration, AppointmentManagement e History

src/components/	Componentes reutilizáveis: Modal, Alerts, AppointmentForm, AppointmentTable, UpcomingAppointments e PrivateRoute
Diretório	Conteúdo
src/contexts/	Contexto de autenticação, que mantém o usuário logado e os estados de carregamento
src/layouts/	Layout do painel, com cabeçalho de navegação e botão de saída
src/api/	Instância do axios configurada com withCredentials
src/types/	Tipos que espelham o contrato da API
src/utils/	Formatação de data e hora para exibição e para os campos de formulário

3.4. Decisões técnicas relevantes

Algoritmo de ordenação (RF14). A ordenação da interface de gestão é feita por um *merge sort* implementado manualmente em backend/src/utils/sorting.ts . A consulta SQL que alimenta esse endpoint (listUnsorted) foi escrita **propositalmente sem cláusula ORDER BY** , de modo que a ordem apresentada ao usuário venha inteiramente do algoritmo implementado, e não do banco de dados. O frontend também não reordena a lista recebida.

Alertas não bloqueiam a gravação (RF17 e RF18). O enunciado determina que o sistema deve *gerar um alerta* em caso de conflito ou indisponibilidade. Optou-se, portanto, por registrar o agendamento e devolver os alertas junto da resposta, deixando a decisão final para o operador. Por consequência, não foi criada nenhuma restrição de exclusão mútua no banco de dados (como EXCLUDE USING gist), o que também permite que a massa de dados de demonstração contenha conflitos propositalis.

Tratamento de data e hora. As colunas de data e hora são do tipo `TIMESTAMP` (sem fuso horário) e representam a *hora local do serviço*: uma faxina às 09:00 é sempre 09:00. Para garantir esse comportamento, o driver pg foi configurado para devolver o valor bruto no formato AAAA-MM-DDTHH:mm:ss , e a API rejeita valores que contenham indicação de fuso horário. Sem esse cuidado, um agendamento editado sem alteração de horário sofreria deslocamento de três horas a cada gravação.

Registro do histórico em transação. A gravação do agendamento e o registro correspondente na tabela de histórico ocorrem dentro da mesma transação de banco. Ou as duas operações são confirmadas, ou nenhuma é — o que impede que exista agendamento sem rastro no histórico.

4. Modelo de Dados

O banco de dados faxina_db é composto por sete tabelas. O Diagrama Entidade-Relacionamento é entregue separadamente, em arquivo de imagem, conforme o item 2 do enunciado.

Tabela	Finalidade	Registros
usuario	Operadores da empresa que acessam o sistema. Armazena o hash bcrypt da senha e o perfil (ADMIN ou OPERADOR)	3

cliente	Pessoas físicas e jurídicas que solicitam o serviço de faxina	3
endereço	Endereços vinculados a um cliente, onde o serviço será executado	3
profissional	Profissionais de limpeza que executam os serviços	3
disponibilidade	Janelas semanais de atendimento de cada profissional (dia da semana e faixa de horário)	3
agendamento	Agendamentos de faxina, com cliente, profissional, endereço, tipo e intervalo de data e hora	4
historico_agendamento	Registro de cada criação, edição e exclusão, com o operador responsável e a data da operação	4

Convenções e restrições

- A coluna `disponibilidade.dia_semana` segue a convenção do `EXTRACT(DOW)` do PostgreSQL: **0 = domingo** até **6 = sábado**. A convenção está documentada no próprio schema por meio de `COMMENT ON COLUMN`.
- Os tipos enumerados `perfil_usuario`, `tipo_faxina` e `tipo_acao` restringem os valores aceitos nas respectivas colunas.
- `historico_agendamento.agendamento_id` usa `ON DELETE SET NULL`: quando um agendamento é excluído, o registro histórico permanece, apenas perdendo a referência. Isso preserva a rastreabilidade exigida pelo enunciado.
- Restrições `CHECK` garantem que a hora de término seja posterior à de início, tanto em `agendamento` quanto em `disponibilidade`.
- Índice `idx_agendamento_prof_inicio` sobre `(profissional_id, data_hora_inicio)` apoia a verificação de conflito, executada a cada gravação.

Massa de dados de demonstração. Os agendamentos do script de população contêm conflitos propositalmente, para que os alertas dos requisitos RF17 e RF18 possam ser demonstrados sem necessidade de cadastro prévio. Os detalhes estão descritos na seção 6.3.

5. Lista de Requisitos Funcionais

Foram identificados 20 requisitos funcionais. A coluna *Origem* indica o item do enunciado que originou cada requisito;

Código	Nome	Descrição	Origem
--------	------	-----------	--------

RF01	Autenticar usuário	O sistema deve permitir que o operador acesse o sistema informando e-mail e senha. As credenciais são validadas contra a tabela <code>usuario</code> , comparando a senha informada com o hash bcrypt armazenado.	Item 4
RF02	Informar motivo da falha de autenticação	Em caso de falha no acesso, o sistema deve exibir o motivo da falha e manter o usuário na tela de autenticação, permitindo nova tentativa.	Item 4.1
RF03	Exibir nome do usuário logado	A interface principal deve exibir o nome do operador autenticado.	Item 5.1.1
RF04	Efetuar logout	O sistema deve oferecer um meio de encerrar a sessão, redirecionando o usuário à tela de autenticação.	Item 5.1.2
RF05	Acessar cadastro de agendamento	A interface principal deve oferecer um meio de acessar a interface de cadastro de agendamento.	Item 5.1.3
RF06	Acessar gestão de agendamentos	A interface principal deve oferecer um meio de acessar a interface de gestão de agendamentos.	Item 5.1.4
RF07	Listar agendamentos automaticamente	Ao acessar a interface de cadastro, o sistema deve carregar automaticamente, em formato de tabela, os agendamentos existentes no banco de dados.	Item 6.1.1
RF08	Buscar agendamentos por termo	O sistema deve oferecer um campo de busca que, após inserção e confirmação do termo, atualize a listagem exibindo apenas os registros correspondentes. A busca abrange nome do cliente, nome do profissional, tipo de faxina, observações, bairro e cidade.	Item 6.1.2
RF09	Cadastrar agendamento	O sistema deve permitir a inserção de um novo agendamento no banco de dados, informando cliente, endereço, profissional, tipo de faxina, data e hora de início e término, e observações.	Item 6.1.3
RF10	Editar agendamento	O sistema deve permitir a alteração dos dados de um agendamento existente.	Item 6.1.4

Código	Nome	Descrição	Origem
RF11	Excluir agendamento	O sistema deve permitir a exclusão de um agendamento existente, solicitando confirmação prévia do operador.	Item 6.1.5
RF12	Validar dados informados	O sistema deve validar os dados preenchidos na criação e na alteração de agendamentos, exibindo alertas em caso de ausência de preenchimento ou de dado inválido. As validações incluem campos obrigatórios, formato de data e hora, término posterior ao início, data não retroativa e pertencimento do endereço ao cliente selecionado.	Item 6.1.6
RF13	Retornar à interface principal	A interface de cadastro deve oferecer um meio de retornar à interface principal do sistema.	Item 6.1.7
RF14	Listar agendamentos ordenados por algoritmo	A interface de gestão deve apresentar os agendamentos em ordem cronológica ou alfabética, utilizando um algoritmo de ordenação implementado no sistema. Foi adotado o merge sort, com três critérios selecionáveis: data, nome do cliente e nome do profissional.	Item 7.1.1
RF15	Selecionar agendamento e definir movimentação	O sistema deve permitir selecionar o agendamento que sofrerá movimentação e escolher entre marcação de faxina residencial ou comercial, ou ainda a alocação de outro profissional.	Item 7.1.2
RF16	Informar data e horário da movimentação	O sistema deve permitir que o operador informe a data e o horário do agendamento que está sendo realizado.	Item 7.1.3
RF17	Alertar conflito de horário	A cada gravação de agendamento, o sistema deve verificar automaticamente se o profissional já possui outro serviço no mesmo intervalo e, em caso positivo, exibir alerta ao operador.	Item 7.1.4

RF18	Alertar indisponibilidade do profissional	A cada gravação de agendamento, o sistema deve verificar automaticamente se o horário solicitado está dentro da janela de disponibilidade cadastrada para o profissional no dia da semana correspondente e, caso não esteja, exibir alerta ao operador.	Item 7.1.4
RF19	Alertar agendamentos próximos	O sistema deve destacar na interface principal os agendamentos prestes a serem realizados, considerando uma janela de 72 horas a partir do momento do acesso.	Desafio

Código	Nome	Descrição	Origem
RF20	Registrar histórico de operações	O sistema deve registrar um histórico completo de cada agendamento, identificando o cliente, o profissional responsável, o operador que realizou a ação, o tipo de ação (criação, edição ou exclusão) e a data da operação.	Desafio

6. Instruções de Execução

Passo 1 — Criar e popular o banco de dados

Com o PostgreSQL 18 em execução, executar o script de criação a partir da raiz do projeto:

```
psql -U postgres -f backend/db/faxina_db.sql
```

O script remove a base existente, recria a base `faxina_db`, cria os tipos, as tabelas e os índices, e insere a massa de dados. Caso exista alguma sessão conectada à base (por exemplo, uma janela do pgAdmin), o comando de remoção falhará; basta encerrar a conexão e executar novamente.

Passo 2 — Configurar as variáveis de ambiente

Na pasta `backend`, copiar o arquivo `.env.example` para `.env` e preencher os valores conforme o ambiente local, em especial a senha do PostgreSQL:

Variável	Finalidade
PORT	Porta da API (padrão 5000)
CORS_ORIGIN	Endereço da interface autorizado a consumir a API
CHAVE_ACESSO	Chave secreta de assinatura do token de acesso
COOKIE_NOME	Nome do cookie que transporta o token
DB_HOST , DB_PORT	Endereço e porta do servidor PostgreSQL
DB_USUARIO , DB_SENHA	Credenciais de acesso ao banco
DB_NOME	Nome da base de dados (<code>faxina_db</code>)

Passo 3 — Iniciar a API

Na pasta `backend`: `npm install` e, em seguida, `npm run dev`. A API passa a responder em `http://localhost:5000`.

Passo 4 — Iniciar a interface

Na pasta `frontend`: `npm install` e, em seguida, `npm run dev`. A interface fica disponível em `http://localhost:3000`.

A interface deve necessariamente ser servida na porta **3000**, pois é o endereço autorizado na configuração de CORS da API. Executá-la em outra porta faz com que todas as requisições sejam bloqueadas pelo navegador.

Passo 5 — Acessar o sistema

Abrir `http://localhost:3000` no navegador e autenticar com uma das credenciais relacionadas na seção 8.

7. Credenciais de Acesso

O script de população cria três operadores. As senhas são armazenadas exclusivamente como hash bcrypt com fator de custo 12; os valores em texto abaixo são fornecidos apenas para avaliação do sistema.

E-mail	Senha	Perfil	Nome
gustavo@faxinaja.com.br	admin123	ADMIN	Gustavo
mariana.duarte@faxinaja.com.br	operador123	OPERADOR	Mariana Duarte
rafael.nunes@faxinaja.com.br	operador123	OPERADOR	Rafael Nunes

Os hashes podem ser regerados, caso as senhas sejam alteradas, por meio do comando `npm run hash` executado na pasta backend .